

Reviewer Pack — Minimal Rank of Multivariate Generalized Polynomials Bounds ...

Entry 05eaaff1dd91 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 10:18:15 UTC

Minimal Rank of Multivariate Generalized Polynomials Bounds Frege Proof Depth

Entry ID: 05eaaff1dd91 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 10:18:15 UTC

1. Conjecture

Field A (mathematical branch): Multivariate Polynomial Theory (Generalized Polynomials) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

{'text': 'For every n-ary Boolean function f, the minimal rank of its multivariate generalized polynomial representation over a finite field is upper-bounded by the Frege proof depth of f.', 'equation': ' $\text{rank}(f) \leq D_{\text{Frege}}(f)$ '}

Rationale (proposer's reasoning):

{'text': 'Multivariate generalized polynomials provide an algebraic representation that captures the structure of Boolean functions. A lower bound on their rank could reveal a new structural barrier for Frege proof complexity, which is central to understanding the strength of propositional proof systems.', 'equation': ''}

Taxonomy category: Frege_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7a1e747884ed94cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the generated n-ary Boolean functions (where $n \leq 40$) and using at least 30 random seeds, the minimal rank of the multivariate generalized polynomial representation is less than or equal to the Frege proof depth by an aggregate metric mean of 3 or less.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (multivariate generalized polynomials) AND (Frege proof complexity) - (minimal rank) AND (Boolean function representation) AND (finite field) - (polynomial theory) AND (upper bound) AND (Frege proof depth)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def multivariate_generalized_polynomial(f, n):
        # Simplified representation using a dictionary
        poly = {}
        for i in range(len(f)):
            term = []
            for j in range(n):
                if (i >> j) & 1:
                    term.append(f'x{j}')
                else:
                    term.append('~x{j}')
            poly[''.join(term)] = f[i]
        return poly

    def frege_proof_depth(poly, n):
        # Simplified estimation of Frege proof depth
        max_depth = 0
        for term in poly:
            depth = len(term.split('x')) - 1
            if depth > max_depth:
                max_depth = depth
        return max_depth

    def min_rank(poly):
        # Simplified calculation of minimal rank (number of terms)
```

```

        return len(poly)

n = random.randint(5, 40)
f = generate_boolean_function(n)
poly = multivariate_generalized_polynomial(f, n)
depth = frege_proof_depth(poly, n)
rank = min_rank(poly)

return {
    "metric_name": "rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= depth,
    "counterexample": "" if rank <= depth else f"Rank {rank} > Depth {depth}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank:.2f} std=NA support_fraction={support_fraction:.2f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank > Depth\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11934
2	propose	ollama_remote	glm4:latest	0	0	20066
3	preregistration	ollama_remote	glm4:latest	0	0	5858
4	novelty	ollama_remote	glm4:latest	0	0	4720
5	novelty	ollama_remote	glm4:latest	0	0	7437
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12953
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10758
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9185
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8256
10	judge	ollama_remote	glm4:latest	0	0	21428

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112594 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/05eaaff1dd91.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/05eaaff1dd91.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/05eaaff1dd91.tar.gz` (if generated)